배포 · DNS 운영 가이드

포트폴리오(enaeble.co.kr)와 사이드 프로젝트 서버를 혼자서 배포·관리하기 위한 정리 · 2026년 10월 기준

> **이 문서 보는 법**
> 각 서비스마다 ① **처음 배포하기** → ② **환경변수/비밀키** → ③ **도메인 연결** → ④ **자주 쓰는 명령어** 순서로 정리했습니다. 대시보드 메뉴 이름은 서비스 업데이트로 조금씩 바뀔 수 있으니, 메뉴가 안 보이면 비슷한 이름을 찾아보세요.

목차

1. 전체 구성 한눈에 보기
2. Cloudflare Pages (포트폴리오 프론트)
3. Pages Functions (문의 메일 API)
4. Cloudflare Workers (대시보드 서버)
5. Streamlit Community Cloud
6. Fly.io (남은 Node 서버용)
7. DNS 기본과 Cloudflare 설정
8. 비밀키 보관 위치
9. 문제가 생겼을 때
10. 명령어 모음

1. 전체 구성 한눈에 보기

프로젝트	올린 곳	비용	잠자기	배포 방식
포트폴리오 화면	Cloudflare Pages	무료	없음	GitHub `main` 푸시 시 자동
포트폴리오 문의 API	Pages Functions (같은 Pages 프로젝트)	무료	없음	프론트와 함께 자동
대시보드 서버	Cloudflare Workers	무료	없음	Workers Builds(GitHub 연동) 또는 `wrangler deploy`
Streamlit 앱	Streamlit Community Cloud	무료	약 12시간 미접속 시	GitHub 푸시 시 자동
남은 Node 서버 2개	미정 (Fly.io 추천)	약 $2/개/월	설정으로 끔	GitHub Actions

> **어디에 올릴지 고르는 기준**
> - **화면만 있음** (정적 사이트) → Cloudflare Pages
> - **REST API만 있음** (요청 → 응답으로 끝남) → Pages Functions 또는 Workers (무료, 잠자기 없음)
> - **WebSocket·실시간** (Socket.IO, 협업 편집 등) → Fly.io 같은 상시 실행 서버
> - **Streamlit** → Community Cloud (무료) 또는 Fly.io

2. Cloudflare Pages 포트폴리오 프론트

① 처음 배포하기

1. Cloudflare 대시보드 → **Workers & Pages** → **Create** → **Pages** 탭 → **Connect to Git**
2. GitHub 저장소 선택 (예: `E-nae/portfolio2026`), Production branch는 `main`
3. 빌드 설정 입력

항목	값 (portfolio2026 기준)	설명
Framework preset	Next.js (Static HTML Export)	없으면 None으로 두고 아래 값 직접 입력
Root directory	`views`	package.json과 `functions/`가 있는 폴더. **틀리면 API가 동작하지 않음**
Build command	`npm run build`	
Build output directory	`out`	`next.config.ts`의 `output: 'export'` 결과 폴더

Next.js에서 정적 배포를 하려면 `next.config.ts`에 아래 설정이 있어야 합니다.

```
const nextConfig = {
  output: 'export',            // 정적 파일로 빌드 (결과: out/)
  images: { unoptimized: true } // 정적 배포에서는 이미지 최적화 서버가 없음
};
```

② 환경변수

Pages 프로젝트 → **Settings** → **Variables and Secrets** → Production에 추가합니다.

종류	예시	언제 쓰이나
`NEXT_PUBLIC_`로 시작하는 값	`NEXT_PUBLIC_RECAPTCHA_SITE_KEY`	**빌드할 때** 화면 코드에 그대로 박힘. 브라우저에 공개돼도 되는 값만
비밀키 (Secret 타입)	`RESEND_API_KEY`, `RECAPTCHA_SECRET`, `CONTACT_RECEIVER_EMAIL`	Functions가 **요청을 처리할 때** 읽음. 화면에는 절대 노출 안 됨

> **환경변수를 추가·수정한 뒤에는 반드시 다시 배포해야 반영됩니다.**
> Deployments → 최신 배포의 ⋯ → **Retry deployment** (또는 아무 커밋이나 푸시)

③ 도메인 연결 (enaeble.co.kr)

1. Pages 프로젝트 → **Custom domains** → **Set up a custom domain**
2. `enaeble.co.kr` 입력 → 도메인이 Cloudflare DNS에 있으면 레코드가 **자동으로** 만들어짐
3. `www.enaeble.co.kr`도 같은 방법으로 하나 더 추가
4. 상태가 **Active**가 되면 끝 (몇 분 ~ 최대 수십 분)

자동으로 만들어지는 레코드는 대략 `CNAME enaeble.co.kr → portfolio2026.pages.dev` (주황 구름, Proxied) 형태입니다. 직접 지우지 마세요.

④ 배포 확인

- Deployments 탭에서 각 커밋의 빌드 로그와 미리보기 주소(`xxxx.portfolio2026.pages.dev`)를 볼 수 있습니다.
- 예전 배포로 되돌리기: Deployments → 원하는 배포의 ⋯ → **Rollback to this deployment**
- 빌드가 Node 버전 문제로 실패하면 환경변수에 `NODE_VERSION = 22` 추가 후 재배포

3. Pages Functions 문의 메일 API

Pages 프로젝트 루트(`views/`) 안의 `functions/` 폴더에 파일을 두면, **파일 경로가 곧 API 주소**가 됩니다. 프론트와 같은 도메인이라 CORS 설정이 필요 없습니다.

파일	주소
`views/functions/api/contact.ts`	`https://enaeble.co.kr/api/contact`
`views/functions/api/hello.ts` (예시)	`https://enaeble.co.kr/api/hello`
`views/functions/api/users/[id].ts` (예시)	`/api/users/123` → `params.id`

기본 형태

```
// 함수 이름이 HTTP 메서드를 정함: onRequestGet, onRequestPost, onRequest(전부)
export const onRequestPost = async ({ request, env }) => {
  const body = await request.json();
  // 비밀키는 env.이름 으로 읽음 (process.env 아님!)
  const res = await fetch('https://api.example.com', {
    headers: { Authorization: `Bearer ${env.SOME_API_KEY}` },
  });
  return Response.json({ success: true });
};
```

> **Express와 다른 점**
>
> - `process.env` 대신 `env.이름` 으로 비밀키를 읽습니다.
> - 요청마다 따로 실행되므로 **메모리에 값을 저장해 둘 수 없습니다** (예: express-rate-limit 같은 메모리 기반 제한). 요청 수 제한이 필요하면 Cloudflare WAF의 Rate limiting rule을 씁니다.
> - Node 전용 라이브러리 일부는 동작하지 않을 수 있습니다. 외부 API는 SDK 대신 `fetch` 로 직접 부르는 게 가장 안전합니다.

로컬에서 테스트하기

`next dev` 로는 Functions가 실행되지 않습니다. 빌드한 뒤 wrangler로 띄워야 합니다. 로컬용 비밀키는 `views/.dev.vars` 에 둡니다 (Git에 올라가지 않음).

```
# views/.dev.vars
RESEND_API_KEY=re_xxxxxxxx
RECAPTCHA_SECRET=xxxxxxxx
CONTACT_RECEIVER_EMAIL=you@example.com
```

```
cd views
npm run build
npx wrangler pages dev out          # http://localhost:8788 에서 화면 + API 함께 실행
```

배포된 API가 살아있는지 확인

```
curl -X POST https://enaeble.co.kr/api/contact \
  -H 'Content-Type: application/json' \
  -d '{"email":"a@b.c","message":"hi","captchaToken":"fake"}'
```

응답	의미
`403 봇 요청으로 판단되었습니다`	**정상**. 함수가 동작 중 (가짜 캡차라 막힌 것)
`405` 또는 HTML 페이지	함수가 배포되지 않음 → Root directory가 `views`인지, `functions/`가 커밋됐는지 확인
`500 메일 전송 실패`	Resend 키나 보내는 도메인 인증 문제 → 7장 Resend 항목 확인

로그 보기

Pages 프로젝트 → Deployments → 배포 선택 → **Functions** 탭 → **Real-time Logs** → Begin log stream.
`console.log` / `console.error` 내용이 여기 찍힙니다.

4. Cloudflare Workers 대시보드 서버

Pages Functions와 같은 엔진이지만, 프론트와 분리된 **독립 API 서버**로 쓸 때 사용합니다.

① 처음 만들기

```
npm create cloudflare@latest my-api   # 템플릿 선택: Hello World / Hono 등
cd my-api
npx wrangler login                    # 브라우저에서 Cloudflare 로그인
npx wrangler dev                      # 로컬 실행 (http://localhost:8787)
npx wrangler deploy                   # 배포 → https://my-api.<계정>.workers.dev
```

② GitHub 자동 배포 연결

1. Workers & Pages → 해당 Worker → **Settings** → **Build** → **Connect** (Git repository)
2. 저장소·브랜치 선택, Root directory는 `wrangler.jsonc` (또는 `wrangler.toml`)가 있는 폴더
3. 이후 `main`에 푸시하면 자동 배포

③ 비밀키

```
npx wrangler secret put DATABASE_URL   # 입력창에 값 붙여넣기
npx wrangler secret list               # 등록된 이름 확인 (값은 안 보임)
```

대시보드에서는 Worker → Settings → **Variables and Secrets**. 로컬 개발용 값은 프로젝트 루트의 `.dev.vars`에 둡니다. Workers는 비밀키 변경 시 바로 반영됩니다.

④ 도메인 연결 (예: api.enaeble.co.kr)

1. Worker → Settings → **Domains & Routes** → **Add** → **Custom domain**
2. `api.enaeble.co.kr` 입력 → DNS 레코드와 인증서가 자동 생성됨

> **프론트와 도메인이 다르면 CORS 설정이 필요합니다.** 응답 헤더에 허용할 프론트 주소를 넣어야 브라우저가 막지 않습니다.
>
> ```
> return new Response(body, { headers: {
>   'Access-Control-Allow-Origin': 'https://enaeble.co.kr',
>   'Access-Control-Allow-Methods': 'GET,POST,OPTIONS',
>   'Access-Control-Allow-Headers': 'Content-Type',
> }});
> ```
>
> Hono를 쓴다면 `app.use('*', cors({ origin: ['https://enaeble.co.kr'] }))` 한 줄로 됩니다.

⑤ 로그

```
npx wrangler tail                     # 실시간 로그
```

무료 플랜 한도 (대략)

- 하루 요청 10만 건, 요청당 CPU 시간 10ms

- WebSocket·실시간 기능은 Durable Objects가 필요 (무료 플랜에서도 일정량 사용 가능)

5. Streamlit Community Cloud

① 처음 배포하기

1. `share.streamlit.io` 접속 → GitHub 계정으로 로그인
2. **Create app** → **Deploy a public app from GitHub**
3. Repository, Branch(`main`), Main file path(예: `app.py`) 입력
4. **App URL**에 원하는 이름 입력 → `https://이름.streamlit.app`
5. **Advanced settings**에서 Python 버전 선택, Secrets 입력 → **Deploy**

저장소 루트(또는 앱 파일과 같은 폴더)에 `requirements.txt`가 있어야 패키지가 설치됩니다.

```
# requirements.txt 예시
streamlit
pandas
requests
```

② 비밀키

앱 화면 오른쪽 아래 **Manage app** → ⋮ → **Settings** → **Secrets**에 TOML 형식으로 입력합니다.

```
API_KEY = "xxxxxxxx"

[database]
url = "postgres://..."
```

```
import streamlit as st
st.secrets["API_KEY"]
st.secrets["database"]["url"]
# 최상위 값은 os.environ["API_KEY"] 로도 읽을 수 있음
```

로컬에서는 `.streamlit/secrets.toml`에 같은 내용을 두고, 이 파일은 `.gitignore`에 꼭 추가하세요.

③ 알아둘 점

- **Render에 따로 배포할 필요 없음.** 단, Streamlit이 별도 API 서버(FastAPI 등)를 호출한다면 그 서버는 따로 띄워야 합니다.
- **잠자기:** 약 12시간 동안 접속이 없으면 잠들고, 방문자가 "Yes, get this app back up" 버튼을 눌러 깨웁니다.
- **커스텀 도메인 미지원:** `이름.streamlit.app` 주소만 쓸 수 있습니다.
- **포트폴리오에 넣기:** iframe으로 넣을 때 주소 뒤에 `?embed=true`를 붙이면 메뉴 없이 깔끔하게 보입니다.
- 앱이 이상하면 Manage app → **Reboot app**. 로그도 Manage app 패널에서 볼 수 있습니다.

6. Fly.io 남은 Node 서버용 · 참고

WebSocket을 쓰는 등 Workers로 옮기기 어려운 서버를 **잠자기 없이** 저렴하게 돌릴 때 씁니다. 카드 등록이 필요하고, 256MB 머신 1개에 월 약 $2입니다.

① 처음 배포하기

```
brew install flyctl
fly auth login
cd 내-서버-폴더
fly launch                # 앱 이름 입력, 지역은 nrt(도쿄) 선택 → fly.toml 생성
fly scale count 1         # 기본으로 2대가 만들어질 수 있음 → 1대로 줄여 비용 절반
```

② fly.toml 핵심 설정 (잠자기 끄기)

```
app = "my-server"
primary_region = "nrt"

[http_service]
  internal_port = 3000          # 서버가 listen 하는 포트와 같아야 함
  force_https = true
  auto_stop_machines = "off"    # 잠자기 끄기
  auto_start_machines = true
  min_machines_running = 1

[[vm]]
  memory = "256mb"
  cpu_kind = "shared"
  cpus = 1
```

Node 서버는 `process.env.PORT` (또는 3000)에서 `0.0.0.0` 으로 listen 해야 합니다. WebSocket은 별도 설정 없이 동작합니다.

③ 비밀키 · 배포 · 로그

```
fly secrets set RESEND_API_KEY=xxxx DATABASE_URL=xxxx   # 설정 즉시 재시작
fly secrets list
fly deploy                # 수동 배포
fly logs                  # 실시간 로그
fly status                # 머신 상태
fly apps open             # 브라우저로 열기
```

④ GitHub 푸시로 자동 배포

1. 토큰 발급: `fly tokens create deploy -x 999999h`
2. GitHub 저장소 → Settings → Secrets and variables → Actions → **New repository secret** → 이름 `FLY_API_TOKEN`, 값은 위 토큰
3. 저장소에 `.github/workflows/fly-deploy.yml` 추가

```
name: Fly Deploy
on:
  push:
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    concurrency: deploy-group
    steps:
      - uses: actions/checkout@v4
      - uses: superfly/flyctl-actions/setup-flyctl@master
      - run: flyctl deploy --remote-only
        env:
          FLY_API_TOKEN: ${{ secrets.FLY_API_TOKEN }}
```

⑤ 도메인 연결 (예: chat.enaeble.co.kr)

```
fly certs add chat.enaeble.co.kr     # 필요한 DNS 레코드를 알려줌
```

1. Cloudflare DNS에 `CNAME chat → my-server.fly.dev` 추가
2. **Proxy status는 회색 구름(DNS only)**으로 설정 — 주황색이면 인증서 발급이 실패할 수 있음
3. `fly certs show chat.enaeble.co.kr`로 인증서가 발급됐는지 확인

7. DNS 기본과 Cloudflare 설정

enaeble.co.kr의 DNS는 Cloudflare에서 관리합니다. 대시보드 → 도메인 선택 → **DNS** → **Records**.

레코드 종류

타입	뜻	예시
A / AAAA	도메인 → IP 주소 (IPv4 / IPv6)	`A @ → 1.2.3.4`
CNAME	도메인 → 다른 도메인 (별명)	`CNAME www → portfolio2026.pages.dev`
TXT	소유 인증, 메일 보안(SPF·DKIM·DMARC)	`TXT @ → "v=spf1 include:..."`
MX	메일을 받을 서버	`MX send → feedback-smtp...`

`@`는 루트 도메인(`enaeble.co.kr`) 자체를 뜻합니다. 이름에 `api`를 쓰면 `api.enaeble.co.kr`가 됩니다.

주황 구름 vs 회색 구름 (Proxy status)

	주황 (Proxied)	회색 (DNS only)
동작	Cloudflare를 거쳐 접속 (캐시·보안·HTTPS 제공)	도메인 주소만 알려주고 직접 접속
쓰는 곳	Pages, Workers, 일반 웹사이트	Fly.io·Render 등 외부 서버, 메일 관련 레코드

> Pages·Workers에서 Custom domain을 추가하면 레코드가 자동으로 만들어지므로, 직접 손댈 일은 외부 서비스(Fly.io 등)를 연결할 때뿐입니다.

Resend 메일 도메인 인증 (contact@enaeble.co.kr)

- Resend 대시보드 → **Domains**에 enaeble.co.kr이 등록되어 있고, Resend가 알려준 TXT·MX 레코드가 Cloudflare DNS에 들어가 있어야 메일이 발송됩니다.
- **이 레코드들은 지우지 마세요.** 이름에 `resend`, `send`, `_dmarc`, `domainkey`가 들어간 레코드가 해당됩니다.
- 새 도메인을 쓸 때는 Resend → Add Domain → 표시되는 레코드를 그대로 Cloudflare에 추가(회색 구름) → Verify.

reCAPTCHA 허용 도메인

Google reCAPTCHA 관리 콘솔(`google.com/recaptcha/admin`) → 사이트 설정 → **도메인**에 실제 접속 주소가 등록돼 있어야 합니다. 새 도메인이나 `pages.dev` 주소에서 테스트하려면 여기에 추가하세요. 등록되지 않은 도메인에서는 문의 폼이 "봇 요청"으로 막힙니다.

요청 수 제한 (선택)

도메인 → **Security** → **WAF** → **Rate limiting rules** → Create rule. 조건: URI Path equals `/api/contact`, 같은 IP에서 일정 횟수 초과 시 Block. 무료 플랜은 규칙 1개까지 가능합니다.

DNS 확인 명령어 (터미널)

```
dig enaeble.co.kr +short              # 루트 도메인이 가리키는 곳
dig CNAME api.enaeble.co.kr +short    # 서브도메인 CNAME 확인
dig TXT enaeble.co.kr +short          # TXT 레코드 확인
curl -I https://enaeble.co.kr         # 사이트 응답 확인 (200이면 정상)
```

DNS를 바꾼 뒤 반영까지 보통 몇 분, 길면 수십 분 걸립니다. Cloudflare 안에서만 바꾼 경우는 대부분 즉시 반영됩니다.

8. 비밀키 보관 위치

비밀키는 **절대 GitHub에 올리지 않습니다.** 아래 위치에만 둡니다.

서비스	배포 환경	로컬 개발
Pages Functions (문의 API)	Pages → Settings → Variables and Secrets	`views/.dev.vars`
Pages 화면 (`NEXT_PUBLIC_`)	Pages → Settings → Variables and Secrets	`views/.env`
Workers	`wrangler secret put` 또는 대시보드	`.dev.vars`
Streamlit	Manage app → Settings → Secrets	`.streamlit/secrets.toml`
Fly.io	`fly secrets set`	`.env`
GitHub Actions	저장소 Settings → Secrets → Actions	-

위 로컬 파일들이 `.gitignore`에 들어 있는지 확인하는 명령어:

```
git check-ignore -v views/.dev.vars views/.env    # 출력이 나오면 무시되고 있는 것 (정상)
```

9. 문제가 생겼을 때

증상	먼저 확인할 것
문의 폼: "메시지를 보내지 못했어요"	3장의 `curl` 테스트 → 응답 코드 표대로 확인
문의 폼: "봇 요청으로 판단되었습니다" (실제 사용자)	Pages에 `RECAPTCHA_SECRET`이 있는지, reCAPTCHA 허용 도메인에 접속 주소가 있는지
환경변수를 바꿨는데 그대로임	Pages는 **재배포**해야 반영 (Retry deployment)
화면에서 `NEXT_PUBLIC_` 값이 비어 있음	Pages 환경변수에 넣었는지 → 넣은 뒤 재배포 (빌드 시점에 박히는 값)
브라우저 콘솔에 CORS 오류	API 서버의 허용 Origin 목록에 프론트 주소(`https://enaeble.co.kr`, `www` 포함)가 있는지
Pages 빌드 실패	Deployments → 빌드 로그 확인. Root directory, Node 버전(`NODE_VERSION`)
도메인 접속 안 됨	Custom domains 상태가 Active인지, `dig`로 레코드 확인
Fly 앱이 응답 없음	`fly status`, `fly logs`. `internal_port`와 서버 포트가 같은지
Streamlit 앱 오류	Manage app 패널 로그, `requirements.txt` 누락 여부, Reboot app

서버를 옮긴 뒤 정리 체크리스트

- [ ] 새 주소에서 기능이 정상 동작하는지 확인
- [ ] 프론트에서 호출하는 API 주소·링크를 새 주소로 교체
- [ ] CORS 허용 목록, reCAPTCHA 허용 도메인 갱신
- [ ] 비밀키를 새 서비스에 옮겼는지 확인

- [ ] 이전 서비스(Render 등) 삭제 — 남겨두면 배포 실패 알림이 오거나 요금이 나갈 수 있음
- [ ] 저장소에서 쓰지 않는 서버 코드 정리, README 갱신

10. 명령어 모음

Git

```
git status                                # 바뀐 파일 확인
git add 파일들 && git commit -m "메시지"
git push origin main                      # 푸시 = Pages/Streamlit/Actions 자동 배포 시작
git log --oneline -5                      # 최근 커밋
```

포트폴리오 (views 폴더)

```
npm run dev                               # 화면만 확인
npm run build                             # 정적 빌드 → out/
npx wrangler pages dev out                # 화면 + 문의 API 로컬 실행
```

Cloudflare Workers

```
npx wrangler login
npx wrangler dev                          # 로컬 실행
npx wrangler deploy                       # 배포
npx wrangler secret put 이름              # 비밀키 등록
npx wrangler tail                         # 실시간 로그
npx wrangler deployments list             # 배포 이력
npx wrangler rollback                     # 직전 배포로 되돌리기
```

Fly.io

```
fly launch                                # 최초 설정
fly scale count 1                         # 머신 1대로
fly deploy                                # 배포
fly secrets set 이름=값                   # 비밀키
fly logs                                  # 로그
fly status                                # 상태
fly certs add 도메인                      # 커스텀 도메인 인증서
fly tokens create deploy -x 999999h       # GitHub Actions용 토큰
```

확인용

```
curl -I https://주소                       # 사이트 응답 코드
dig 도메인 +short                          # DNS 레코드
curl -X POST https://enaeble.co.kr/api/contact \
  -H 'Content-Type: application/json' \
  -d '{"email":"a@b.c","message":"hi","captchaToken":"fake"}'    # 403이면 정상
```

> **비용 요약** — Cloudflare Pages·Functions·Workers, Streamlit Community Cloud는 무료. Fly.io를 쓰면 256MB 머신 1대 당 월 약 $2 (512MB 약 $3). 요금은 바뀔 수 있으니 결제 전에 각 서비스의 Pricing 페이지를 확인하세요.